

Transformers

Catarina Pereira & Marco
Steenbergen



Overview

1. Self-attention
2. Encoders and Decoders
3. BERT
4. Transfer Learning

Self-Attention

What We Have Done So Far

- We have turned text into numeric representations called embeddings.
- This was a milestone, but it still has limitations:
 - Sometimes limited context (Word2Vec)
 - Always static
- What we want is dynamic embeddings that take in a lot of context.
- Enters the idea of **attention**.

Attention Is All You Need

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

- Let each token attend to each other token, regardless of position.
- Use that attention to update a set of embeddings → bring in context.
- Run many of these attention mechanisms in parallel → multi-head attention.

The Intuition

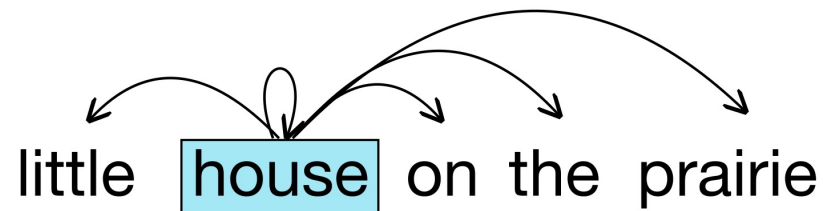
- Start with an initial embedding:
 - E.g., “house” will be placed near “home”, “estate”, etc.
- Bring in context:
 - E.g., “little house” will move the embedding of house away from “estate”.
- Bring in even more context:
 - E.g., “little house on the prairie” (image) will move the embedding of “house” completely away from abodes and toward “tv series” and “entertainment”.



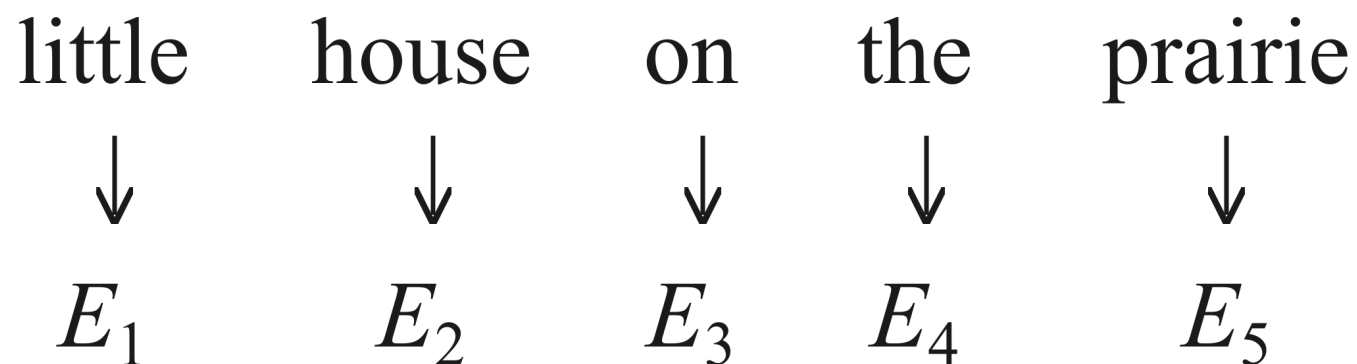
Self-Attention

The purpose of self attention

How much attention should “house” pay to each word in the sequence?



Initial Embedding Space



The embeddings are high-dimensional vectors.

The Updating Mechanism

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{K}^\top \mathbf{Q}}{\sqrt{d}} \right) \mathbf{V}$$

A Thought Experiment

- Imagine we are interested in the way in which adjectives alter the meaning of words.
- In the phrase “little house on the prairie” this amounts to asking how “little” affects the meaning of “house”.
- Now imagine the noun “house” asking whether any relevant adjectives might speak to it.
- We call this the **query**.
- It is encoded as another vector but typically has a lower dimensionality than the original embeddings.
- We scale down using the product $\mathbf{W}_Q \mathbf{E}_i = \mathbf{Q}_i$

The Experiment Continued

- For the earlier query, “little” should respond with “yes, I am a relevant adjective.”
- More precisely, it should have a high activation.
- The answers are called the **keys**.
- They are transformations of the embeddings: $\mathbf{W}_K \mathbf{E}_j = \mathbf{Q}_j$
- Note that every word can be a key, including the query word itself.

Bringing Keys and Queries Together

- How well does a key speak to a query?
- The answer lies in the dot product of key and query.
- Its value shows the relevance of keys to queries.

$$\mathbf{K}^\top \mathbf{Q} = \begin{pmatrix} & \text{little} & \text{house} & \cdots & \text{prairie} \\ \text{little} & \mathbf{K}_1 \mathbf{Q}_1 & \mathbf{K}_1 \mathbf{Q}_2 & \cdots & \mathbf{K}_1 \mathbf{Q}_5 \\ \text{house} & \mathbf{K}_2 \mathbf{Q}_1 & \mathbf{K}_2 \mathbf{Q}_2 & \cdots & \mathbf{K}_2 \mathbf{Q}_5 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{prairie} & \mathbf{K}_5 \mathbf{Q}_1 & \mathbf{K}_5 \mathbf{Q}_2 & \ddots & \mathbf{K}_5 \mathbf{Q}_5 \end{pmatrix}$$

Some Bells and Whistles

- What we have now seen is the crux of it all.
- However, Vaswani et al. (2017) proposed some important modifications:
 - Stabilize through scalar multiplication by $1/\sqrt{d}$
 - Turning attention into probabilities using softmax → **attention mechanism** → tells us with context words contribute to the representation of the target word.
 - Rescaling back to the original dimensionality of the embeddings: this is where the **values** become relevant, and they entail another weight matrix.

A Note of Caution

- In this explanation, we assumed full control over the queries.
- In reality, we do not control what is being attended to.
- The various weight matrices are optimized for prediction → interpretation comes later.

Why This Is Revolutionary

- Capable of incorporating massive amounts of context, without introducing bottlenecks like vanishing gradients.
- Fast because of much lower complexity than alternatives.
- But it turns out that attention is not all we need.

Positional Encoding

Almost There and Yet So Far Away

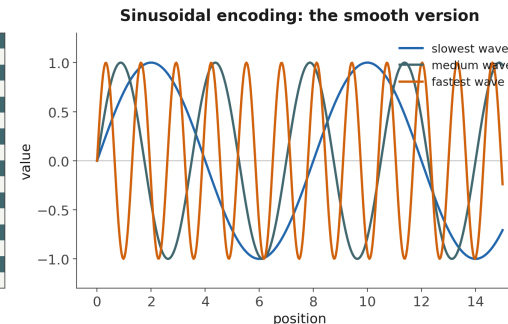
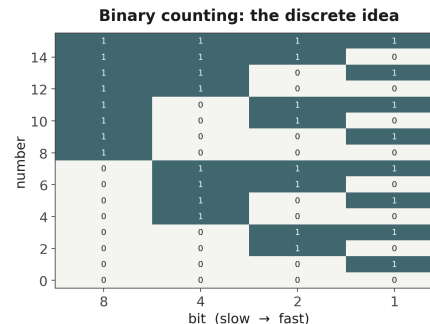
- Attention gets rid of sequence.
- But does not order still matter?
 - “Dog bites man” and “man bites dog” are made of the same words but mean very different things.
- The trouble is that transformers process all tokens in parallel. There is no first, no next, no last.
- The fix is to hand each token a description of *where* it sits: a **positional encoding**.

The Naïve Fix

- You might tag each token with its index: 0 for "little", 1 for "house", etc.
- But a neural network cannot do much with a raw integer:
 - *Unbounded scale*: Position 500 adds "500" to an embedding whose entries are around 1. Position swamps meaning, and training destabilizes.
 - *No generalization to new lengths*: If training never saw a sentence longer than 40 tokens, the network has no idea what position 4 000 means.
 - *Normalization does not save it*: Rescaling bounds the range but now position 5 means "halfway" in a 10-word sentence and "5%" in a 100-word one. The same index no longer means the same thing.
- What we really want is an encoding that is bounded, works at any length, and lets the network recover relative distance ("three tokens back") easily.

A Better Idea: Count Like a Clock

- How do we label arbitrarily many positions with a small, bounded alphabet? We use rhythms of different speeds.
- In binary counting the last bit flips every step (fast), the next every two steps, the next every four (slow). Each number is a unique pattern of fast and slow bits.
- A clock does the same with hands: the second hand races, the hour hand crawls; read together they pin down the moment exactly.
- Positional encoding is just the smooth version of this: replace the on/off bits with sine and cosine waves.



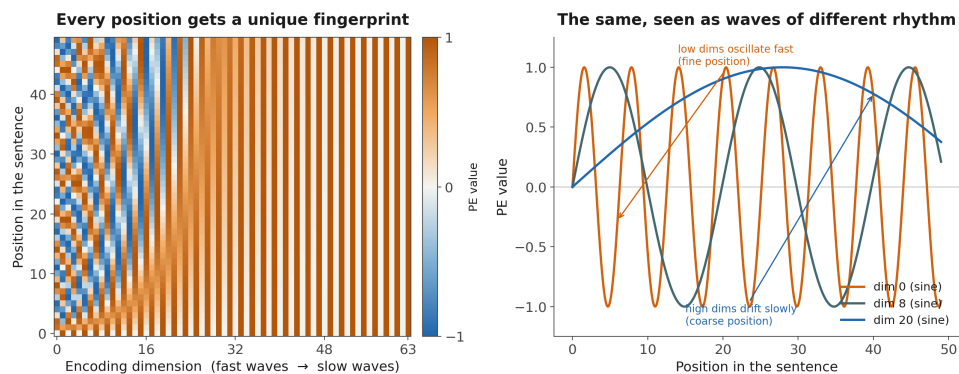
Positional Encoding Done Right

- Vaswani (2017) give each position a vector whose entries are sine and cosine waves:

$$\begin{aligned} \text{PE}_{\text{pos}, 2i} &= \sin\left(\frac{\text{pos}}{10000^{2i/d_m}}\right) \\ \text{PE}_{\text{pos}, 2i+1} &= \cos\left(\frac{\text{pos}}{10000^{2i/d_m}}\right) \end{aligned}$$

- pos – the token's position 0, 1, ...
- $d[m]$ – the model dimension: the length of the vector, chosen equal to the embedding so we can add the two.
- i – indexes pairs of entries. Each pair shares one frequency: an even slot uses sin; the odd slot next to it uses cos.
- The denominator is the wavelength dial. Small $i \Rightarrow$ denominator near 1 $\Rightarrow$ a fast wave; large $i \Rightarrow$ denominator up to 10 000 $\Rightarrow$ a slow wave.

Reading the Formula



- Left: each row is one position's encoding—its fingerprint. No two row are alike, yet nearby rows look similar.
- Right:
 - Low dimensions oscillate fast
 - They resolve fine position, token vs, neighbor
 - High dimensions oscillate slowly
 - They mark coarse position, start vs. end
- Together, the fast and slow waves give every position a distinct, bounded signature – exactly the clock, drawn out.

Illustration: “prairie” at Position 5

- Take the sentence “little house on the prairie”.
- With a tiny model of six entries, we have three frequency pairs: 0, 1, and 2.
- Consequently,

$$i = 0 : \quad 10000^{0/6} = 1 \quad \Rightarrow \quad \sin(5), \cos(5) \quad = \quad -0.96, \quad 0.28$$

$$i = 1 : \quad 10000^{2/6} \approx 21.5 \quad \Rightarrow \quad \sin(0.232), \cos(0.232) \quad = \quad 0.23, \quad 0.97$$

$$i = 2 : \quad 10000^{4/6} \approx 464 \quad \Rightarrow \quad \sin(0.011), \cos(0.011) \quad = \quad 0.01, \quad 1.00$$

- We add these to the embeddings.

Why Sines and Cosines?

The pairing is the clever part. For one frequency, ω , a shift of k positions is a rotation of the (sin,cos) pairs.

$$\begin{bmatrix} \sin \omega(\text{pos} + k) \\ \cos \omega(\text{pos} + k) \end{bmatrix} = \begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix} \begin{bmatrix} \sin \omega \text{pos} \\ \cos \omega \text{pos} \end{bmatrix}$$

The rotation depends only on the offset k , never on the absolute position.

So "three tokens back" is a fixed linear operation – precisely the kind of thing the linear weight projections can learn.

That is what integers could not offer: with sinusoids, relative position is baked in, ready for attention to exploit.

Determinism versus Learning

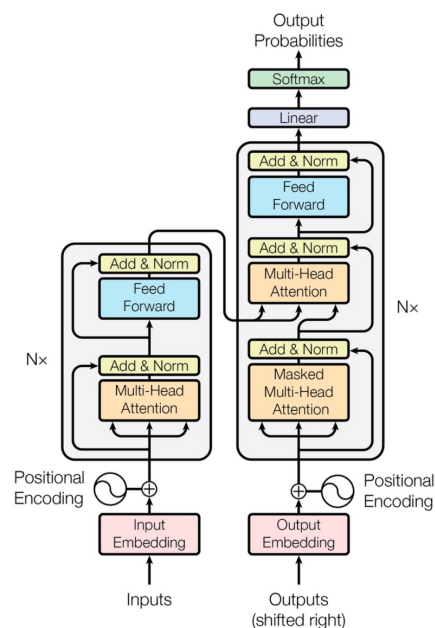
- Vaswani et al. (2017) fixed the positional encodings—there was nothing to train—and they extrapolate to sequences longer than those seen during training.
- Models like BERT instead learnt a position vector from the data.
 - Often a little better distribution for the training data
 - However, they cannot reach beyond the lengths they were trained on
- Recent developments have moved to *relative* and *rotary* (e.g., RoPE):
 - These carry the shift = rotation idea into the attention score

Encoders and Decoders

Definitions

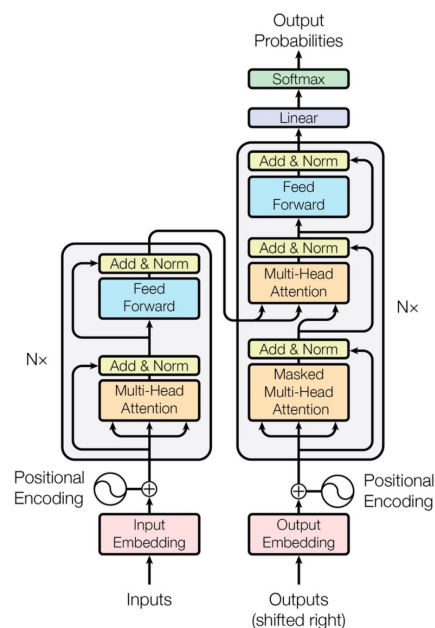
- Encoders = representation builders.
- Decoders = sequence generators.
- Vaswani et al. (2017) built both into a single architecture.
- Modern LLMs often separate these.

The Full Model: Encoding



- Input words become embeddings with positional encodings added on top.
- Results pass through a stack of 6 layers:
 - Self-attention
 - FFN
 - Wrapped into add & norm: a residual shortcut that lets information skip past the step + normalization for stability
- Output = context-rich vector.

The Full Model: Decoding



- Another stack:
 - Masked self-attention:
 - Output words attend to each other, but each position can only look backwards
 - Encoder decoder cross-attention:
 - Questions come from the decoder
 - Answers come from the encoder's output
 - FFN as in encoder
- Each step again wrapped into add and norm.
- Final layer + softmax turns the decoder's output into a probability distribution over the vocabulary.

BERT

What Is BERT?

- BERT = Bidirectional Encoder Representation from Transformers.
- An encoder-only algorithm developed by Devlin et al. (2019).
- Several important innovations:
 - Bidirectionality
 - Masked language modeling
 - Next sentence prediction

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal

A Stack of Encoders

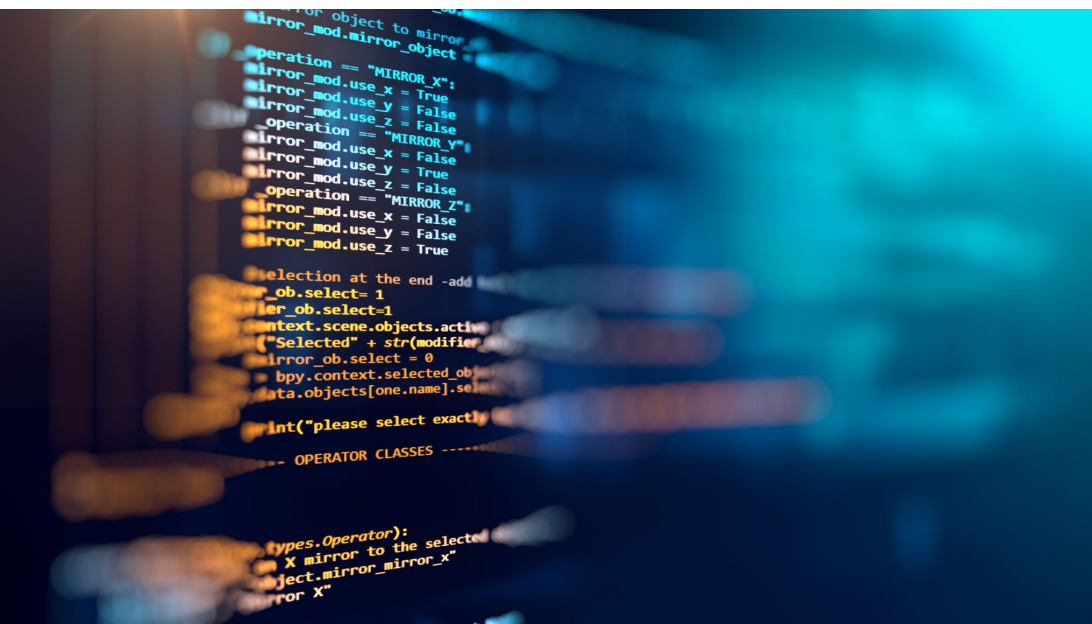
Attribute	BERT-Base	BERT-Large
Encoder layers	12	24
Hidden size	768	1024
Attention heads	12	16
FFN inner size	3072	4096
Parameters	110M	340M
Max input length	512 tokens	512 tokens

From Text to Tokens

- BERT does not embed whole words; the WordPiece tokenizer first splits text into subword units:
 - A fixed vocabulary of 30,522 pieces, learned from the training corpus
 - Frequent words survive intact ("dog"); rare words decompose ("playing" → "play" + "##ing")
 - No word is ever out of vocabulary; in the worst case it falls apart into characters
- Special tokens frame every input:
 - [CLS] opens the sequence; [SEP] closes each sentence
- Key point: WordPiece decides which units get a vector. It supplies no vectors itself.

Three Embeddings, One Input

- Every token ID indexes a row of a learned embedding matrix ($30,522 \times 768$ in BERT-base).
- Two further learned embeddings are added to it:
 - A position embedding: where in the sequence the token sits (0-511)
 - A segment embedding: sentence A or sentence B
- The sum is layer-normalized; this is what self-attention first sees.
- Before pre-training, all three weight matrices are random draws from a truncated normal ($SD = 0.02$); training turns them into meaningful vectors.
- Note: unlike the sinusoids of Vaswani et al. (2017), BERT learns its positional encodings.



Bidirectionality

- Early transformers were unidirectional:
 - Read text from left to right
 - Only earlier tokens can inform the understanding of current tokens
- BERT is bidirectional:
 - Queries can use any key, regardless of whether it came earlier or later

What Comes Out of BERT?



- Each token leaves the final encoder layer with a contextual embedding.
 - The same word gets different vectors in different contexts ("bank of England" vs. "bank of the river")
- The final vector of [CLS] acts as a summary of the whole sequence.
 - This is where classification heads attach during fine-tuning
- Alternative: average (mean-pool) the token vectors into a single text representation.
- These contextual vectors are the "updated embeddings" that fine-tuning builds on.

Masked Language Modeling (MLM)

- A pre-training method.
- Some tokens are randomly masked during training (hidden from the model).
 - In BERT: 15% of tokens are selected
 - Of these, 80% become [MASK], 10% become a random token, 10% stay unchanged – because [MASK] never occurs at fine-tuning time
- The model is tasked with predicting the original token.
- BERT is taught deep contextual relationships, because words must be inferred purely from context.

Next Sentence Prediction (NSP)

- Originally, BERT was trained on a task called next sentence prediction.
- Task:
 - Given two sentences determine whether second sentence follows logically from the first.
- BERT learns sentence-level coherence, allowing an understanding of larger text spans.

Different BERT Variants

Attribute	BERT	RoBERTa	DistilBERT	ALBERT
Parameters	110M	125M	66M	12M
Speed	Medium	Medium	Fast	Fast
Tokenizer	WordPiece	BPE	WordPiece	SentencePiece
Training corpus	Book corpus + Wikipedia	As in BERT + CC-News + OpenWebText	As in BERT	As in BERT
GLUE	84.5	88.5	82.2	89.4

Some Notes about the Table

- Tokenizers:
 - BPE = byte-pair-encoding; starts with individual characters and merges them.
 - WordPiece: Similar to BPE but with a probabilistic model; more effortful; works well with English.
 - SentencePiece: general tokenizer that includes BPE and other methods; often used in multilingual contexts.
- GLUE = General Language Understanding Evaluation:
 - A performance metric for LLMs, especially encoders.
 - Based on performance on numerous tasks such as grammar/syntax, sentiment detection, paraphrase detection, semantic similarity detection, natural language inference, logic, pronoun resolution in ambiguous contexts.

What the Variants Changed

- RoBERTa (Liu et al., 2019):
 - Drops NSP, masks dynamically, trains longer on roughly 10× more data
 - Same architecture as BERT; a better training recipe
- DistilBERT (Sanh et al., 2019):
 - Knowledge distillation: a 6-layer student learns to mimic BERT's outputs
 - 40% smaller and 60% faster, yet retains ~97% of performance
- ALBERT (Lan et al., 2020):
 - Shares parameters across layers and factorizes the embedding matrix
 - Replaces NSP with sentence-order prediction (SOP)

The Wider BERT Family

ELECTRA: replaced-token detection instead of masking; the model learns from every token, not just the 15% that are masked.

Multilingual models:

mBERT (104 languages), XLM-RoBERTa

ManifestoBERTa builds on XLM-RoBERTa

Domain- and task-adapted models:

SciBERT and BioBERT for scientific text; PopBERT builds on a German BERT

Recipe: continued pre-training or fine-tuning on specialist corpora

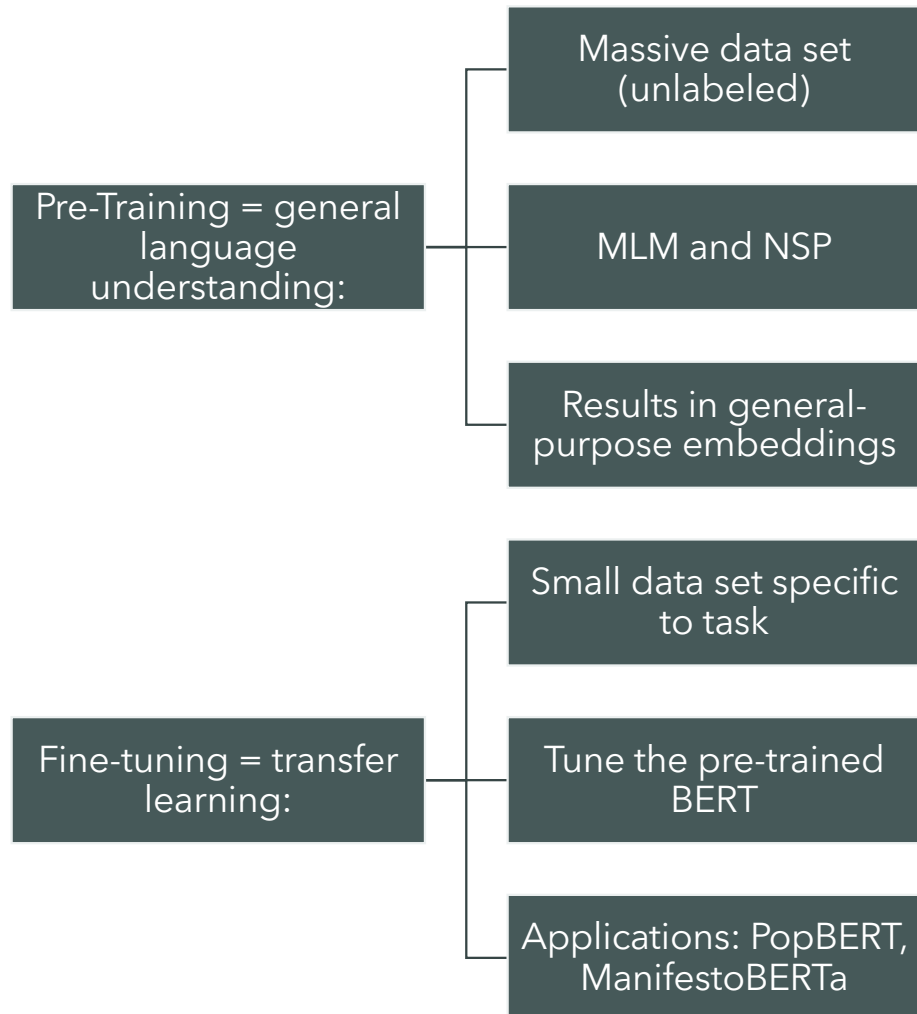
Sentence-BERT (SBERT): fine-tuned to produce sentence embeddings for semantic similarity.

Beyond Classification: BERTopic

- So far we have used BERT to classify; its embeddings also support unsupervised discovery.
- BERTopic (Grootendorst, 2022) is a topic-modeling pipeline built on BERT-family embeddings:
 - Embed each document (Sentence-BERT)
 - Compress the embeddings (UMAP) and cluster them (HDBSCAN)
 - Describe each cluster with class-based TF-IDF → interpretable topic words
- Note: this is an application on top of BERT, not a new variant.
- BERT itself is not fine-tuned; it only supplies the embeddings;
 - Any embedding model can be swapped in
- Contrast with LDA: contextual embeddings capture meaning, not just word co-occurrence.

Fine-Tuning an Encoder

Two-Step Training a la Devlin et al.



Delving Deeper: Different Logics

1. Feature extraction
2. Full fine-tuning
3. Partial unfreezing
4. Parameter-efficient fine-tuning (PEFT)
5. Continued pre-training

Feature Extraction

Run the encoder in inference mode, train only the classifier on top.

Pros: cheapest by far; no risk of catastrophic forgetting; one shared encoder can serve many tasks; embeddings can be pre-computed once, after which training is trivial; works with very little labeled data; fully reproducible.

Cons: lowest performance ceiling because the representations cannot adapt to the domain; particularly problematic when the domain's vocabulary differs from the pre-training corpus.

Full Fine-Tuning

The recipe proposed by Devlin et al.: add a domain-specific head and adjust all encoder weight, using a small learning rate ($\sim 2e-5$).

Pros: Usually the strongest performer.

Cons: one complete model copy per task; unstable on small datasets; risk of over-fitting.

Partial Unfreezing

Train the head + top k layers OR unfreeze layers progressively, starting from the top. Apply layer-wise learning decay.

Pros: a genuine compromise; protects lower layers; cheaper.

Cons: Additional hyper-parameters in the form of k and the learning rate schedule; performance not as good as full unfreezing.

PEFT

Keep the pre-trained weights frozen but insert or overlay small trainable components.

Many variants:

- Adapters (Houlsby et al. 2019): small bottleneck layers inside each encoder block; only these are trained (~1-3%).
- LoRA (Hu et al. 2021): learn low-rank update matrices for the attention weights; after training merge those into the original weights to speed up inference.
- BitFit: train only the bias terms—is competitive on many GLUE-style tasks.

Pros: near-full fine-tuning performance at a fraction of the cost; one frozen encoder + a drawer full of task modules; more stable on smaller datasets.

Cons: a small performance gap to full fine-tuning is possible; new hyper-parameters.

Continued Training

No fine-tuning at all but continued pre-training on a large unlabeled domain corpus (Gurugangan et al.'s DAPT/TAPT). Then apply any other options. Use for BioBERT.

Pros: adapts the model to domain vocabulary and style; needs no labels.

Cons: requires a large domain corpus and serious computational power.